



TM6 Review Guide for Test 6 on 04.30

Completion requirements

Association Rule Learning

- Terms to know:
 - **association rule** (or **rule**)
 - **market basket analysis**
 - **item set** (or **itemset**)
 - **support** (for an association rule)
 - **Apriori algorithm**
- You should know that **confidence**, **lift**, and **leverage** are important metrics used by the **Apriori algorithm** to learn association rules. However, you do not need to memorize their formulas.
- You should be prepared to explain what association rule learning is, the problem it solves, and the algorithm used to find rules.
- You should be able to compare and contrast association rule learning with other unsupervised methods such as K-means. (Association rule learning finds patterns of co-occurrence among items, while clustering finds groups of examples based on similarity.)

Neural Networks

- Terms to know:
 - **neural network** (also known as **artificial neural network** [ANN] or **multilayer perceptron** [MLP])
 - **units, neurons, nodes**
 - **layers, input layer, output layer, hidden layer**
 - **weights, bias** (intercept term)
 - **activation function**
 - **forward propagation, feed-forward network** [FF]
 - **deep network**
- Make sure you know how a single unit works: given input values, weights, and an activation function, you should be able to compute the output.
- You should understand how data passes through the entire network to produce a final prediction. There may be a problem similar to the example shown in the slides, where you are asked to compute the network output from given inputs.
- Each unit in a neural network is very similar to a **generalized linear model (GLM)**: it computes a weighted sum of its inputs, applies an optional bias (intercept), and then

passes the result through an activation function. (A logistic regression model, for example, is a simple GLM with a sigmoid activation.)

- You should have an intuitive understanding of what an **activation function** does and why it is necessary in neural networks.
- You should be familiar with several activation functions, their tradeoffs, and when they are appropriate. (You do not need to memorize the formulas.)
 - **Identity function** (used for regression in the output layer)
 - **Heaviside step function** (used for perceptrons; not generally used for MLPs because it is not differentiable)
 - **Sigmoid function** (commonly used for binary classification outputs)
 - **Hyperbolic tangent (tanh)** (squashes outputs between -1 and 1; centered at 0)
 - **ReLU (Rectified Linear Unit)** (most common for hidden layers in modern networks)
 - **Leaky ReLU** (a variation of ReLU that allows a small, nonzero gradient when the unit is inactive)
 - **Softmax** (used for multi-class classification in the output layer)
- Keep in mind that the **input layer** does not contain units that "work": input nodes have no weights or activation functions; they are simply inputs.
- You should understand that each unit implements a simple function, and the network as a whole implements a complex function. With enough hidden units, a network can approximate **any** continuous or finite discrete function. Neural networks are known as **universal approximators**.

Training a Neural Network

- Terms to know:
 - **cost function**
 - **gradient**
 - **method of gradient descent**
 - **backpropagation** (error backpropagation)
 - **epoch**
 - **batch, minibatch, and stochastic gradient descent**
- Make sure you are familiar with the general way gradient descent works. You should know that it is an **optimization method** that relies on the **gradient** (a generalization of a **derivative** for multivariable functions).
- Be prepared to explain what backpropagation does and how it relates to gradient descent.
- Understand that under the hood, modern neural networks rely on **vectorized operations** (fast matrix calculations) using highly optimized linear algebra libraries. "Nodes" are a teaching construct; actual implementations use matrix operations.

- You should know the differences between batch, minibatch, and stochastic gradient descent:
 - **Batch gradient descent (BGD)**: computes the gradient using the entire training set, then updates the weights all at once. (One pass = one epoch. Training typically requires many epochs.)
 - **Stochastic (online) gradient descent (SGD)**: updates the weights after **each** training example (one row at a time).
 - **Minibatch gradient descent (MGD)**: updates the weights after a small random subset ("minibatch") of examples from the training set.

Convolutional Neural Networks (CNNs)

- Terms to know:
 - **convolutional neural network (CNN)**
 - **sparse vs. fully connected network**
 - **convolutions**
 - **pooling** (e.g., **max pooling**, **average pooling**, etc.)
 - **convolve**, **kernel**, **stride**
- You should understand how image data is represented when passed into a neural network:
 - For standard MLPs, image data is typically **flattened** (each pixel becomes an input value).
 - For CNNs, the image is passed as a **2D (or 3D) array**, preserving its width, height, and channel structure to maintain spatial relationships.
- You should understand how CNNs differ from ordinary multilayer perceptrons (MLPs).

Final Note

- For all machine learning methods, you should be able to discuss the tradeoffs between different methods and identify the parameters and hyperparameters of each method.

Last modified: Tuesday, May 13, 2025, 3:00 PM